Topics Covered:

1) OSI
2) Threeway Handshake
3) Vpn
4) Routing protocols
5) Switch and routers
6) Tcp vs Udp
7) Testing fundamentals for python core
8) C Programs

OSI (Open Systems Interconnection) Model

Bottom to Top (1 → 7): Please **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way
Top to Bottom (7 → 1): All **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing

Layer	Name	What It Does	Key Concept / Protocol	Netgear Relevance
7	**Application**	The interface human users interact with (web browsers, email apps).	HTTP, HTTPS, FTP, DNS	Router web dashboards, DNS lookups
6	**Presentation**	Translates, encrypts, and compresses data so the receiver can read it.	SSL/TLS, JPEG, ASCII	HTTPS encryption (TLS)

5	**Session**	Opens, maintains, and closes the communication link between devices.	NetBIOS, RPC, Sockets	Keeping active connections open
4	**Transport**	Manages end-to-end delivery, flow control, and error recovery.	TCP (Reliable), UDP (Fast)	Port numbers (e.g., Port 80, 443)
3	**Network**	Figures out the best path to route data packets across different networks.	IP Addresses (IPv4, IPv6), ICMP	Routers operate mainly here
2	**Data Link**	Handles physical node-to-node transfer on the same local network.	MAC Addresses, Ethernet	Switches operate mainly here
1	**Physical**	Transmits raw binary bits (0s and 1s) over physical media.	Cables, Wi-Fi radio signals	Ethernet cables, Wi-Fi antennas, Hubs

Threeway handshake

At a high level, **TCP (Transmission Control Protocol)** is the protocol responsible for making sure data gets delivered reliably, completely, and in the right order. Before any actual data (like a webpage or video) travels between two devices, TCP establishes a verified connection first using a three-step greeting: the **Three-Way Handshake**.

The Everyday Analogy: A Phone Call

Imagine you're calling a friend on a noisy line:

1. **You:** "Hey, can you hear me?" (*SYN*)
2. **Friend:** "Yes, I hear you! Can you hear me?" (*SYN-ACK*)
3. **You:** "Yes, loud and clear!" (*ACK*)

Now both of you know that both sides can speak and hear, and you can safely start your actual conversation.

Step-by-Step Technical Breakdown

1.Step 1: SYN (Synchronize): Client → Server.

The client (your laptop) sends a small packet with the **SYN flag** turned on. It says, *"I want to open a connection, and here is my starting sequence number (m)."*

2.Step 2: SYN-ACK (Synchronize-Acknowledge): Server → Client.
The server receives the request and replies with both the **SYN** and **ACK flags** set. It says, *"I got your request (ACK = m + 1), and here is my starting sequence number (n)."*

3.Step 3: ACK (Acknowledge): Client → Server.
The client sends a final confirmation packet back with the **ACK flag** set. It says, *"I got your server setup details (ACK = n + 1). We are officially connected!"*

- **Where does this happen?** At **Layer 4 (Transport Layer)** of the 7-layer OSI model.
- **Why use sequence numbers?** Random **Initial Sequence Numbers (ISNs)** are picked by both sides to track packet order, prevent missing data, and protect against malicious connection hijacking.
- **What happens after the handshake?** Actual payload data is transmitted using flags like PSH (Push) to deliver data immediately.
- **How does it close?** TCP uses a **4-step teardown process** using FIN (Finish) and ACK flags to close the connection gracefully from both directions.
- **What is Wireshark's role?** Wireshark is a packet analyzer. In a Wireshark capture, you can visually identify these exact three packets at the very start of any TCP conversation (like an HTTP or HTTPS request).

VPN'S

What is a VPN?

A **Virtual Private Network (VPN)** is a security service that creates an **encrypted tunnel** between a user's device and a remote server over the internet.

It accomplishes two primary goals:

1. **Confidentiality:** Encrypts data so third parties (ISPs, hackers, advertisers) cannot spy on traffic.
2. **Anonymity:** Hides the user's real public IP address, replacing it with the VPN server's IP address.

How a VPN Works (4 Steps)

1.Establish Connection:
Step 1.
The VPN client software authenticates the user and establishes a secure session with a VPN server.

2.Data Encryption:
Step 2.
Your device encrypts all outgoing network packets using cryptographic algorithms (e.g., AES-256).

3.Tunneling & Masking:
Step 3.
Encrypted data travels through the "tunnel" to the VPN server. The server replaces your real IP address with its own IP address.

4.Decryption & Forwarding:
Step 4.
The VPN server decrypts your data and forwards it to the target website/service. Replies travel back through the exact same secure tunnel.

Types of VPNs

1. By Usage (Deployment)

- **Remote Access VPN:** Connects an individual user to a private corporate network over the internet (essential for remote work).
- **Site-to-Site VPN:** Connects two static networks (e.g., linking a company's branch office to its main headquarters).
- **Mobile VPN:** Designed to keep sessions stable as mobile devices switch between Wi-Fi and cellular networks.

2. By Protocols (Tunneling Tech)

- **OpenVPN:** Popular open-source protocol using SSL/TLS. Highly secure and versatile.
- **WireGuard:** Modern, extremely lightweight, and fast protocol using state-of-the-art cryptography.
- **IKEv2/IPsec:** Very fast and handles network transitions (e.g., switching from Wi-Fi to 4G) seamlessly.
- **L2TP/IPsec & PPTP:** Older protocols. PPTP is obsolete/insecure; L2TP/IPsec is secure but can be slow due to double encapsulation.

Key Trade-offs & Advantages

- **Benefits:** Protects public Wi-Fi traffic, bypasses geo-restrictions, secures remote work access, and prevents ISP throttling.
- **Drawbacks:** Introduces slight speed latency (due to encryption overhead), can be blocked by strict firewalls, and relies on trusting the VPN provider's no-log policy.

WIFI Routing protocols

In Wi-Fi networks and hardware engineering, **routing protocols** can refer to two distinct areas:

1. **Dynamic Routing Protocols** (how routers talk to other routers on an IP network).
2. **Wi-Fi Mesh Routing Protocols** (how wireless access points/nodes seamlessly route traffic to eliminate dead zones—a massive focus for **Netgear** with products like their Orbi system).

Here is a breakdown of both concepts for a Netgear technical interview.

1. Dynamic IP Routing Protocols (Layer 3)

Routers use dynamic routing protocols to automatically discover network paths, build routing tables, and adapt if a cable or link drops.

Static vs. Dynamic Routing

- **Static Routing:** Paths are manually typed in by a network admin. Simple, highly secure, but terrible for scaling or adapting to failures.
- **Dynamic Routing:** Routers run algorithms to dynamically calculate the best path to a destination.

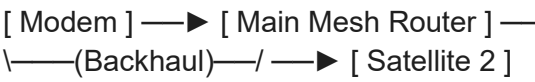
The Two Major Types of Dynamic Protocols

Metric / Feature	Distance-Vector (e.g., RIP)	Link-State (e.g., OSPF)
How it works	Tells its immediate neighbors what it knows ("Routing by rumor").	Map out the **entire network topology** locally using Dijkstra's algorithm.
Metric Used	Hop count (max 15 hops).	Cost (calculated using link speed/bandwidth).
Speed & Convergence	Slow convergence; prone to routing loops.	Fast convergence; highly scalable for large networks.
Analogy	Asking passersby for directions at every intersection.	Using a full GPS map of the entire city.

Key Interview Term: BGP (Border Gateway Protocol) is an Exterior Gateway Protocol (EGP) used to route traffic *between* different Autonomous Systems (the protocol that runs the global internet).

2. Wi-Fi Mesh Routing Protocols (Wireless Focus)

When preparing for Netgear, **Mesh Wi-Fi routing** is a critical topic. Traditional extenders create separate network names (SSIDs) and degrade speed. Mesh systems (like Netgear Orbi) use smart wireless routing to act as a single, unified network.



Key Concepts in Wi-Fi Mesh Routing

1. **IEEE 802.11s Protocol:** The standard for wireless mesh networking. It allows nodes (satellites) to automatically discover each other and route data using protocols like **HWMP (Hybrid Wireless Mesh Protocol)**.
2. **Path Selection:** Mesh nodes constantly measure link quality (signal strength, interference, and latency) to decide whether to send a client's packet directly to the main router or hop through an intermediate satellite.
3. **Dedicated Backhaul (Netgear Orbi Secret Sauce):**
    - **Problem:** Standard mesh nodes share the same Wi-Fi band for client traffic (your phone) and node-to-node communication, cutting speed in half per hop.
    - **Netgear Solution:** Netgear uses **Tri-Band / Quad-Band technology** with a *dedicated wireless channel (backhaul)* reserved exclusively for router-to-satellite communication.

3. Interview Questions & Answers

Q1: "What is the difference between a Wi-Fi Extender and a Mesh Network?"
"An extender acts as a simple relay that broadcasts a separate network (e.g., `Home_EXT`*), requiring manual reconnection and halving bandwidth. A Mesh system uses intelligent routing under a* ***single SSID****, enabling seamless Roaming (802.11k/v/r) and dynamic path selection without speed drops."*

Q2: "How do routers prevent routing loops in Distance-Vector protocols?"
"They use mechanisms like ***Split Horizon*** *(don't advertise a route back out the interface it was learned from),* ***Route Poisoning*** *(setting an unreachable metric like 16 hops), and* ***Hold-Down Timers****."*

Q3: "What happens when a client moves from one mesh satellite to another?"
"The mesh network handles a ***seamless handover*** *using standards like* ***802.11k*** *(assisted roaming queries),* ***802.11v*** *(BSS transition management), and* ***802.11r*** *(fast BSS transition for quick re-authentication)."*

The Analogy: An Office Building

Imagine an office building:

- **The Switch is like the internal mailroom.** It delivers mail between desks inside the same office building.
- **The Router is like the mail truck/post office.** It takes mail sent from inside your office and routes it across town to a completely different company or address (the Internet).

1. What is a Switch? (Internal Communication)

A **switch** connects multiple devices (computers, printers, smart TVs) together within the **same local network (LAN)**.

- **How it works:** Every device has a unique, permanent hardware identifier called a **MAC Address** (like a device's fingerprint). When Computer A wants to send data to Computer B, the switch looks up its **MAC Address Table**, identifies which physical port Computer B is plugged into, and sends the data directly to Computer B.
- **OSI Layer:** Works at **Layer 2 (Data Link Layer)**.
- **Key Role:** Creating a local network and enabling efficient, direct device-to-device communication.

2. What is a Router? (External Communication)

A **router** connects **different networks together**—most commonly connecting your private home/office network to the public Internet (WAN).

- **How it works:** Devices on different networks communicate using **IP Addresses** (logical locations, like mailing addresses). When you try to access google.com, your request leaves your local network. The router inspects the destination IP address, checks its **Routing Table**, and forwards the data along the best path to reach Google's servers.
- **OSI Layer:** Works at **Layer 3 (Network Layer)**.
- **Extra Features:** Home routers usually perform extra tasks like **DHCP** (assigning IP addresses automatically) and **NAT** (translating private local IPs into a single public IP).

Direct Comparison for Interviews

Feature	**Switch**	**Router**
Primary Job	Connects devices inside *one* network	Connects *multiple* different networks together
OSI Layer	**Layer 2** (Data Link)	**Layer 3** (Network)
Address Used	**MAC Address** (Hardware address)	**IP Address** (Logical address)
Data Format	Data Frames	Data Packets

Scope	Local Area Network (**LAN**)	Wide Area Network (**WAN**) / Internet

Interview Golden Rule: A **switch** uses **MAC addresses** to move traffic inside a network. A **router** uses **IP addresses** to move traffic between networks.

---

1. TCP vs. UDP (Transport Layer Protocols)

Both protocols operate at **Layer 4 (Transport Layer)** of the OSI model. Their main job is to move data between specific applications on two devices using **Port Numbers**.

```
        ┌──────────────────────┐
        │  Transport Layer  │
        └──────────┬───────────┘
           │
    ┌──────────────────┴───────────────────┐
    ▼                ▼
  TCP (Reliable Mail)       UDP (Quick Postcard)
• Connection-oriented      • Connectionless
• Guarantees delivery       • Speed-focused
• Slower, checks errors     • Faster, accepts lost data
```

TCP (Transmission Control Protocol)

* **What it is:** A **connection-oriented** protocol. Think of it like a **phone call**: you must pick up and establish a connection before you start talking.
* **How it works:**
1. **3-Way Handshake:** Before sending data, TCP establishes a connection using 3 steps: `SYN` $\$rightarrow `SYN-ACK` $\$rightarrow `ACK`.
2. **Reliability:** Every data packet gets an acknowledgment (`ACK`). If a packet is lost, TCP automatically resends it.
3. **Order:** Packets are given sequence numbers so they can be reassembled in the exact right order.

* **Use Cases:** Web browsing (`HTTP`/`HTTPS`), Email (`SMTP`), File downloads (`FTP`) — places where missing data breaks everything.

UDP (User Datagram Protocol)

* **What it is:** A **connectionless** protocol. Think of it like sending **postcards**: you drop them in the mailbox without checking if the recipient is ready, and you don't know if they arrived.
* **How it works:** No handshake, no retransmissions, no packet ordering, and tiny header overhead. Data is sent as fast as possible.
* **Use Cases:** Live video streaming, Online gaming, Voice calls (`VoIP`), `DNS` lookups — places where speed matters more than losing an occasional packet.

Key Interview Comparison

Feature	TCP	UDP
Connection	Connection-Oriented (3-Way Handshake)	Connectionless
Reliability	**Guaranteed** (Retransmits lost data)	**Best-effort** (Lost data is dropped)
Data Order	Strict in-order delivery	Unordered
Header Size	20 to 60 bytes (heavier)	8 bytes fixed (lightweight)
Speed	Slower (due to tracking & acknowledgments)	Faster (minimal overhead)

---

2. IP Addressing (Network Layer)

An **IP (Internet Protocol) Address** works at **Layer 3 (Network Layer)**. It serves as a logical mailing address for a device on a network.

IPv4 vs. IPv6

* **IPv4:** 32-bit address, written in dotted-decimal format (e.g., `192.168.1.10`). Provides roughly 4.3 billion addresses, which have run out globally.
* **IPv6:** 128-bit address, written in hexadecimal (e.g., `2001:0db8:85a3::8a2e:0370:7334`). Introduced to solve the IPv4 address exhaustion problem.

Public vs. Private IP Addresses

* **Private IP:** Used *inside* your home or office local network (LAN). Devices on the Internet cannot see or talk to a private IP directly.
* *Common private ranges:* `10.x.x.x`, `172.16.x.x` – `172.31.x.x`, `192.168.x.x`.

* **Public IP:** Assigned to your router by your Internet Service Provider (ISP). It is globally unique on the public Internet.

> **How they work together:** **NAT (Network Address Translation)** allows all devices in your home (laptop, phone, smart TV) to share **one public IP address** when talking to the Internet.

Key Concepts You Will Be Asked About

1. **1. Getting an IP Address (DHCP):** Application Layer (UDP 67/68).
When a device connects to Wi-Fi, it doesn't have an IP yet. It uses **DHCP** (Dynamic Host Configuration Protocol) via a 4-step process called **DORA**:

* **D**iscover: Client broadcasts "Is there a DHCP server?"
* **O**ffer: Server replies "Here is an IP you can use."
* **R**equest: Client says "I'll take that IP."
* **A**cknowledgement: Server locks it in.

2. **2. Finding Local Devices (ARP):** Data Link Layer.
To send data inside the local network, your computer knows the target's IP address, but needs its physical **MAC Address**. It uses **ARP** (Address Resolution Protocol):

* Computer broadcasts: *"Who has IP 192.168.1.5?"*
* Target replies: *"That's me, here is my MAC address."*

3. **3. Leaving the Network (Default Gateway):** Network Layer.
If the target IP is outside your local network (e.g., `8.8.8.8`), your computer sends the packets to the **Default Gateway** (your local Router). The router handles forwarding it across the Internet.

---

3. Top 3 Questions to Practice Out Loud

1. **"Why does DNS use UDP for small queries but TCP for zone transfers?"**
* *Answer:* Simple DNS lookups are small and fast, so **UDP** avoids the delay of establishing a connection. Zone transfers carry large amounts of critical data between servers, so **TCP** ensures no data is dropped or corrupted.

2. **"What happens when you type `google.com` into a browser?"**

* *Answer:* Browser uses **DNS** (usually UDP) to get Google's IP address. Then, it starts a **TCP 3-Way Handshake** (`SYN`, `SYN-ACK`, `ACK`) with that IP to establish a secure `HTTPS` connection.

3. **"What is the difference between MAC and IP addresses?"**
* *Answer:* **MAC** is a permanent 48-bit physical hardware ID burnt into the network card (Layer 2). **IP** is a logical 32-bit or 128-bit network location assigned dynamically (Layer 3).

Testing fundamentals
At Netgear, software testing isn't just about general web apps—it heavily focuses on **networking hardware, firmware, router APIs, and embedded devices**. Testing roles at Netgear typically combine **Software Testing Fundamentals** with **Python Automation**. Here is a breakdown of what you need to know for the interview.

Part 1: Core Software Testing Fundamentals

Interviewers will expect you to understand testing methodologies and how they apply to products like routers, switches, and firmware.

1. Types of Testing

- **Functional Testing:** Verifying that a specific feature works as expected (e.g., verifying that changing a Wi-Fi password in the Netgear router portal updates the setting).
- **Non-Functional Testing:** Testing performance, load, stress, and security (e.g., how many concurrent devices can connect to a Wi-Fi access point before it drops packets).
- **Regression Testing:** Re-running previous tests after a new firmware build to ensure recent code changes haven't broken existing features.
- **Smoke Testing (Sanity Check):** A quick set of high-level tests run right after a new firmware build is deployed to check if the device is stable enough for deep testing.

2. Test Cases & Bug Lifecycle

- **Test Case Anatomy:** Title, Prerequisites, Test Steps, Expected Result, and Actual Result.
- **Bug Lifecycle:** `New` $\rightarrow$ `Assigned` $\rightarrow$ `In Progress` $\rightarrow$ `Fixed/Resolved` $\rightarrow$ `Retested` $\rightarrow$ `Closed`.
- **Flaky Tests:** A test that sometimes passes and sometimes fails without any changes to the code under test. You fix this by using explicit waits instead of hardcoded delays (`time.sleep`).

Part 2: Python for Test Automation

Python is the primary language used for test scripts at Netgear due to its vast ecosystem of networking libraries.

1. Essential Python Built-ins

- **Data Structures:** Know when to use **Lists** (ordered, mutable), **Tuples** (immutable), and **Dictionaries** (key-value pairs, crucial for JSON API responses).

- **Exception Handling (`try-except-finally`):** Critical for preventing test automation scripts from crashing when a router loses connection or times out.

```
Python
import

try
                    "http://192.168.1.1/api/status"      5
    assert                  200
except
        "Router API timed out!"
```

2. Testing Framework: `pytest`

`pytest` is the most popular framework for Python test execution. Key concepts interviewers ask about:

- **Assertions:** Simple `assert expected == actual` statements.
- **Fixtures (`@pytest.fixture`):** Reusable setup and teardown code. (e.g., establishing an SSH connection to a Netgear switch before tests run, and closing it after).
- **Parametrization (`@pytest.mark.parametrize`):** Running the same test logic with multiple inputs (e.g., testing 10 different IP configurations on a router).

```
Python
import

# Testing invalid IP formats using pytest parametrization
@pytest.mark.parametrize("ip_address", ["256.0.0.1", "abc.def.ghi.jkl", "192.168.1"])
def test_invalid_ip_validation
    assert                  False
```

Part 3: Python Network Testing Libraries (Netgear Focus)

In a Netgear interview, showing awareness of network-specific Python tools will set you apart.

Library	Primary Use Case
`requests`	Sending HTTP/REST API calls to router administrative panels or cloud APIs.
`paramiko` / `netmiko`	SSH/Telnet connections to directly run CLI commands on switches and routers.

scapy	Crafting, modifying, and injecting raw network packets (TCP, UDP, ICMP) for deep packet testing.
socket	Low-level Python module for testing port connections (e.g., checking if Port 80 or 443 is listening).

Top 3 Scenario Questions to Practice

1. **"How would you automate testing the login page of a Netgear router?"**
    - *Answer:* Perform API automation via `requests` sending `POST` credentials, check status codes, handle session tokens/cookies, and validate fallback scenarios on invalid attempts. Use `pytest` for assertion checks.
2. **"Why avoid using `time.sleep()` in test scripts?"**
    - *Answer:* `time.sleep()` slows execution unnecessarily or causes flaky tests if the device takes slightly longer than expected. Instead, use polling loops or `WebDriverWait` (for UI).
3. **"How do you handle test data management?"**
    - *Answer:* Keep environment variables, IP addresses, and user credentials out of test scripts by storing them in external `.env`, JSON, or YAML configuration files.

Bitwise Manipulation: Turn On, Turn Off, & Check Odd/Even

In C, we use **masks** with bitwise operators to target specific bit positions (where bit 0 is the rightmost/least significant bit):

- **Turn ON a bit (Set):** Use the Bitwise OR operator (`|`).
- **Turn OFF a bit (Clear):** Use the Bitwise AND operator (`&`) with a inverted mask (`~`).
- **Check Odd/Even:** Check if the least significant bit (bit 0) is set using AND (`& 1`). If bit 0 is `1`, the number is odd; if `0`, it's even.

```
#include <stdio.stdio.h>
#include <stdio.h>

// Function to print a byte in binary for visual clarity
void printBinary(unsigned char num) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (num >> i) & 1);
    }
```

```
    printf("\n");
}

int main() {
    unsigned char num = 12; // Binary: 0000 1100
    int pos = 1;            // Bit position to alter (0-indexed)

    printf("Original number: %d (Binary: ", num);
    printBinary(num);

    // --- 1. TURN ON A BIT (SET) ---
    // Shift 1 to the left by 'pos' bits, then bitwise OR
    unsigned char turnedOn = num | (1 << pos);
    printf("\nAfter turning ON bit at position %d: %d (Binary: ", pos, turnedOn);
    printBinary(turnedOn);

    // --- 2. TURN OFF A BIT (CLEAR) ---
    // Invert the shifted mask (~), then bitwise AND
    pos = 3; // Let's turn off the bit at position 3
    unsigned char turnedOff = num & ~(1 << pos);
    printf("After turning OFF bit at position %d: %d (Binary: ", pos, turnedOff);
    printBinary(turnedOff);

    // --- 3. ODD OR EVEN USING BITWISE AND ---
    printf("\n--- Odd/Even Check ---\n");
    int testNum = 27;

    if (testNum & 1) {
        printf("%d is ODD (Bit 0 is 1)\n", testNum);
    } else {
        printf("%d is EVEN (Bit 0 is 0)\n", testNum);
    }

    return 0;
}
```

Least Common Multiple (LCM)

The efficient way to calculate the LCM of two numbers a and b is by using their **Greatest Common Divisor (GCD)** through the relationship:

```
#include <stdio.h>

// Helper function to calculate GCD using Euclidean Algorithm
```

```
int findGCD(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// Function to calculate LCM
long long findLCM(int a, int b) {
    if (a == 0 || b == 0) return 0; // LCM with 0 is 0

    // We divide before multiplying to prevent potential integer overflow
    return ((long long)a / findGCD(a, b)) * b;
}

int main() {
    int num1, num2;

    printf("Enter two positive integers: ");
    if (scanf("%d %d", &num1, &num2) == 2) {
        long long lcm = findLCM(num1, num2);
        printf("The LCM of %d and %d is: %lld\n", num1, num2, lcm);
    } else {
        printf("Invalid input.\n");
    }

    return 0;
}
```

ODD or even using bitwise operators : `num & 1`